Taller 2

Pruebas de software

Juan José Jara Calle

Paso 0

Repositorio: https://github.com/JaraCalle/TALLER-2-pruebas-de-software

Paso 1

- ¿Qué parte del sistema no controlamos?

No controlamos la API externa ya que son componentes de software a los cuales no tenemos acceso directo generando dependencias externas

Paso 2

- ¿Por qué la base de datos es una dependencia?

La base de datos es una dependencia ya que al ser un modulo externo al propio código de la app, la base de datos puede fallar, la conexión puede fallar o puede presentar intermitencias. Aunque en este momento sea sqlite local, en algún momento se podría migrar al algo mas externo que me genere latencia o inconvenientes externos a la app. Por lo cual la app depende del correcto funcionamiento de la db

Paso 3

- ¿Quién garantiza que el usuario existe?

En este momento, quien me garantiza la existencia de un usuario es la API externa. Así que la existencia de los usuarios en la app son completamente dependientes de la API, que es quien garantiza la existencia de los mismos

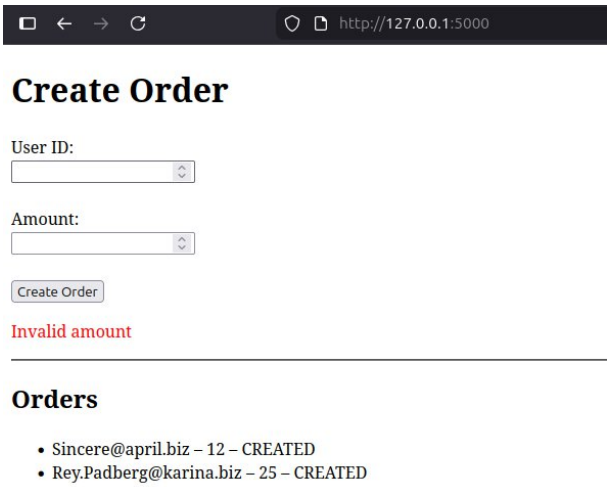
Paso 4

- ¿La lógica sabe de dónde viene el usuario?

No, para la app realmente no es relevante de donde vienen los usuarios. La app lo unico que sabe es que al consultar un usuario al repositorio con un ID, debería retornar algo. Pero el repositorio puede traer los datos de cualquier lugar sin modificar el funcionamiento de lógica de la app.

Paso 5

- Orden creada y datos persistidos



Paso 6

– Salida del test unitario.

```
venv ~/Documents/EAFIT/Taller 2 - Pruebas de software/TALLER-2-pruebas-de-software/orders_app git:(main)  3 files changed, 11 insertions(+), 14 dele
pytest test_unit_real_api.py

========================================================== test session starts ==========================================================
platform linux -- Python 3.13.7, pytest-9.0.2, pluggy-1.6.0
rootdir: /home/jjjara/Documents/EAFIT/Taller 2 - Pruebas de software/TALLER-2-pruebas-de-software/orders_app
plugins: Faker-40.12.0
collected 1 item

test_unit_real_api.py .                                                                                                          [100%]

========================================================== 1 passed in 1.04s ============================================================
```

– Salida del test de integración.

```
venv ~/Documents/EAFIT/Taller 2 - Pruebas de software/TALLER-2-pruebas-de-software/orders_app git:(main)  4 files changed, 13 inse
pytest test_integration_real_api.py -v

========================================================== test session starts ==========================================================
platform linux -- Python 3.13.7, pytest-9.0.2, pluggy-1.6.0 -- /home/jjjara/Documents/EAFIT/Taller 2 - Pruebas de software/TALLER-2-prueba
s-de-software/orders_app/venv/bin/python3
cachedir: .pytest_cache
rootdir: /home/jjjara/Documents/EAFIT/Taller 2 - Pruebas de software/TALLER-2-pruebas-de-software/orders_app
plugins: Faker-40.12.0
collected 1 item

test_integration_real_api.py::test_create_order_integration_real_api PASSED                                                      [100%]

========================================================== 1 passed in 0.93s ============================================================
```

– ¿Este test es siempre confiable? ¿Por qué?

El test es confiable, pero no siempre. Ya que en las ocasiones en las que la db se encuentre caida o hayan inconvenientes para realizar la conexión con la db o con la API externa. El test va a fallar por causas externas que no prueban directamente el funcionamiento lógico de la app

– ¿Cuál es la diferencia entre test_unit_real_api y test_integration_real_api?

La diferencia aunque no en código si tiene una diferencia conceptual. Ya que el test unitario únicamente realiza la prueba de funcionamiento para la creación de una orden. Sin necesidad

de probar nada mas de los demás módulos del código. En cambio, el test de integración se encarga de probar el funcionamiento de la app durante la secuencia de creación de una orden y no solo la función create_order()

Paso 7

- Salida del test unitario.

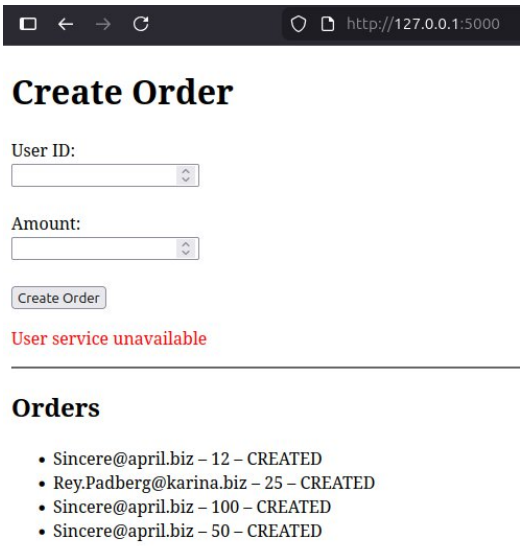
```
venv ~/Documents/EAFIT/Taller 2 - Pruebas de software/TALLER-2-pruebas-de-software/orders_app git:(main)  4 files changed, 14 insertions(+), 20 deletions(-) (1.508s)
pytest test_unit_real_api.py -v

========================================================== test session starts ==========================================================
platform linux -- Python 3.13.7, pytest-9.0.2, pluggy-1.6.0 -- /home/jjjara/Documents/EAFIT/Taller 2 - Pruebas de software/TALLER-2-pruebas-de-software/orders_app/venv/bin/python3
cachedir: .pytest_cache
rootdir: /home/jjjara/Documents/EAFIT/Taller 2 - Pruebas de software/TALLER-2-pruebas-de-software/orders_app
plugins: Faker-40.12.0
collected 1 item

test_unit_real_api.py::test_create_order_with_real_api FAILED                                                                    [100%]

================================================================= FAILURES ==================================================================
_____________________________________________________ test_create_order_with_real_api _____________________________________________________

    def test_create_order_with_real_api():
        db = SessionLocal()
>       order = create_order(1, 100, NullNotifier(), DummyLogger(), db, JsonPlaceholderUserRepository())
                ^^^
```

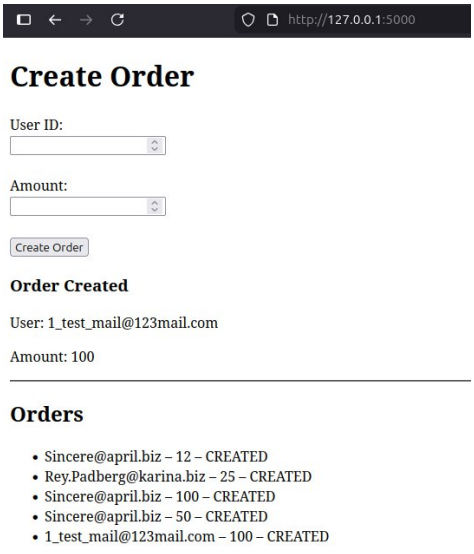


- ¿Qué se rompió realmente?

La conexión con la API externa. Ya que no existia forma de establecer una conexión, lo cual tuvo una implicación directa en que el Order Service aunque realizaba la consulta con el User ID, no obtenía ninguna respuesta

Paso 8

- App funcionando sin API.



Paso 9

- ¿Por qué este sí es un test unitario?

En el anterior test unitario, se estaba utilizando la API real. Lo cual implicaba que se estaba realizando en realidad una prueba de integración. Pero en este test, estamos usando todo de forma local sin ningún tipo de dependencia externa. De esta forma, podemos unicamente el funcionamiento de la funcion create_order() cómo lo debe hacer una prueba unitaria

- Ejecución del test unitario.

```
venv ~/Documents/EAFIT/Taller 2 - Pruebas de software/TALLER-2-pruebas-de-software/orders_app git:(main)  6 files changed, 24 insertions(+), 33 deletions(-) (1.272s)
pytest test_unit_with_stub.py -v

============================================================================================ test session starts ============================================================================================
platform linux -- Python 3.13.7, pytest-9.0.2, pluggy-1.6.0 -- /home/jjjara/Documents/EAFIT/Taller 2 - Pruebas de software/TALLER-2-pruebas-de-software/orders_app/venv/bin/python3
cachedir: .pytest_cache
rootdir: /home/jjjara/Documents/EAFIT/Taller 2 - Pruebas de software/TALLER-2-pruebas-de-software/orders_app
plugins: Faker-40.12.0
collected 1 item

test_unit_with_stub.py::test_create_order_with_stub PASSED                                                                                                                                                [100%]

============================================================================================= 1 passed in 0.38s =============================================================================================
```

Paso 10

- ¿Por qué sigue siendo integración?

Sigue siendo un test de integración ya que dentro del test se está probando el flujo de funcionamiento de la app, lo que integra la lógica de order service, una respuesta de una API (aunque sea fake), y por ultimo también integra el correcto funcionamiento de la db al crear el registro, guardarlo y borrarlo para probar.

- Ejecución del test con fake

```
venv ~/Documents/EAFIT/Taller 2 - Pruebas de software/TALLER-2-pruebas-de-software/orders_app git:(main)  6 files changed, 24 insertions(+), 33 deletions(-) (1.201s)
pytest test_integration_with_fake.py -v

============================================================================================ test session starts ============================================================================================
platform linux -- Python 3.13.7, pytest-9.0.2, pluggy-1.6.0 -- /home/jjjara/Documents/EAFIT/Taller 2 - Pruebas de software/TALLER-2-pruebas-de-software/orders_app/venv/bin/python3
cachedir: .pytest_cache
rootdir: /home/jjjara/Documents/EAFIT/Taller 2 - Pruebas de software/TALLER-2-pruebas-de-software/orders_app
plugins: Faker-40.12.0
collected 1 item

test_integration_with_fake.py::test_create_order_integration_with_fake PASSED                                                                                                                            [100%]

============================================================================================= 1 passed in 0.46s =============================================================================================
```